



FACULTY OF INFORMATION TECHNOLOGY
CITY UNIVERSITY MALAYSIA - CYBERJAYA CAMPUS

ASSIGNMENT 1 - INDIVIDUAL (20%)

SELF-REFLECTIVE REPORT

Object-Oriented Programming Fundamentals in Java

BIT1123 / BISE2093 / DIT1113

PREPARED BY

SALEH ANWAR MOSLEH ANWAR

Student ID: 202504010001

Class Code: BIT1123

Programme: Bachelor of Computer Science (Software Engineering)

NRIC / Passport: K0602248

Lecturer: Sir Nazmirul Izzad Bin Nassir

Submission Date: 21 August 2026

Contents

1. Introduction	2
2. Knowledge Gained from Each Tutorial	3
3. Challenges Encountered	7
4. How the Challenges Were Overcome	8
5. Improvements in Java Programming Skills	8
6. Understanding of Object-Oriented Programming Concepts	9
7. Future Learning Plans	9
8. Conclusion	10
9. GitHub Repository URL	10

1. Introduction

This report reflects on my learning journey through the Object-Oriented Programming tutorials completed for BIT1123. The work in my GitHub repository begins with basic Java syntax and console programs, then develops into class design, encapsulation, inheritance, polymorphism, abstraction, collections, file handling, and a graphical user interface. This progression helped me understand that Java is not only about writing statements in order; it is also about organizing data and behaviour into objects that can work together.

The tutorials also introduced practical software-development habits. I used one repository to store the weekly source files, supporting documentation, and a README. Working with GitHub made me more aware of folder organization, file naming, commit history, and the importance of explaining how a project can be understood by another person. These skills are useful beyond this assignment because they are part of normal professional development work.

Overall learning outcome I progressed from running small console programs to designing reusable class hierarchies, saving data to a file, and responding to user actions in a Swing interface.

2. Knowledge Gained from Each Tutorial

Each tutorial added one layer to my understanding. The repository evidence and main learning focus are summarized below before the detailed reflection.

Week	Repository evidence	Main knowledge gained
1	HelloWorld.java; StudentGrade.java	Java entry point, variables, output, conditions, and initial GitHub workflow
2	main.java; student.java	Classes, objects, fields, constructors, and instance methods
3-4	Person; Student; Lecturer	Inheritance, method overriding, private data, and runtime polymorphism
5	Student; Main; documentation.txt	Encapsulation through private fields, getters, and setters
6	Employee; Lecturer; Main	Parent/child relationships, protected fields, super, and specialization
7	Appliance; Main	Abstract classes, abstract methods, and polymorphic behaviour
8-9	Main.java; task.txt	ArrayList, loops, Scanner, file persistence, and exception handling
10	Questions; QuizBattleGUI	Swing components, event listeners, state changes, and model/UI separation

Week 1 - Getting Started with Java and GitHub

Week 1 gave me the foundation needed for all later tutorials. In HelloWorld.java, I learned the structure of a Java program, especially the class declaration and the public static void main(String[] args) entry point. I used System.out.println() to produce clear console output. StudentGrade.java then introduced variables, data types, string concatenation, and an if/else-if/else decision structure. With a mark of 85, the program assigns grade A and displays a Pass status, which showed me how a program can make a decision from stored data.

This week also introduced GitHub and Codespaces. I learned to create a repository, add Java files, check changes with git status, stage work, commit it, and push it. At first Git commands felt separate from programming, but I began to see the repository as a record of my development progress rather than only a place to upload finished files.

Week 2 - Classes, Objects, Constructors, and Methods

Week 2 moved from standalone statements to an object. The expression `student s1 = new student("Mosleh", 21, 3.8);` creates an instance and passes values to a constructor. I learned how constructor parameters are assigned to the object's name, age, and GPA fields, and how the same object can perform several behaviours through displayInfo(), study(), and takeExam().

The most important lesson was the relationship between a class and an object. The class acts as a blueprint, while s1 is a real object created from that blueprint. Separating the Student data and methods from the main program made the code easier to understand and prepared me for inheritance and encapsulation in the following weeks.

Weeks 3-4 - Inheritance, Overriding, and Polymorphism

The Person, Student, and Lecturer classes showed how related types can share a common design. Person stores a private name and ID, provides getters, and defines introduce(). Student and Lecturer extend Person and call super(name, id) so the parent portion of each object is initialized correctly. Both child classes override introduce() with their own messages. This reduced duplication because the shared identity data remained in one parent class.

Polymorphism became clear in Main.java through declarations such as `Person p1 = new Student("Ali", "S001");`. Although the reference type is Person, p1.introduce() runs the Student version. The same pattern works for Lecturer. This taught me that one parent type can refer to different child objects, while Java selects the correct overridden method at runtime.

Week 5 - Encapsulation and Controlled Access

Week 5 focused on protecting student information. The Student class declares studentID, name, cgpa, and the additional programme field as private. Main.java cannot access these values directly; instead, it uses setter methods to store them and getter methods to read them. I learned that encapsulation keeps an object's internal state under the control of its own class and prevents unrelated code from changing fields without going through an approved method.

The documentation.txt file strengthened this concept because I had to explain why public variables weaken data protection and why getters and setters support validation and maintenance. The current setters assign values directly, but I now understand that validation could be added later. For example, setCGPA() could reject a value below 0.0 or above 4.0 without changing the code that calls the method.

Key reflection Weeks 3-5 changed the way I thought about data. Instead of exposing fields, I began designing clear relationships and controlled access between classes.

Week 6 - Extending an Employee into a Lecturer

Week 6 applied inheritance to a practical employee scenario. Employee is the parent class and stores the shared id and name fields as protected. Lecturer extends Employee and adds subject and department. The Lecturer constructor calls `super(id, name)` before assigning its own fields. This helped me understand constructor chaining: the parent class is responsible for initializing the state it owns, while the child class initializes the details that make it more specialized.

The Lecturer object can call `displayInfo()` even though that method is written in Employee. It also calls `displaySubject()` and `displayDepartment()` from its own class. The output combines inherited and specialized information: employee ID L100, Dr Ahmad, Java Programming, and the Faculty of Information Technology. This was a clear example of code reuse through an is-a relationship: a lecturer is an employee with additional responsibilities.

Week 7 - Abstraction and Polymorphic Appliances

The smart-home activity introduced abstraction through the abstract Appliance class. Appliance contains the common brand field and the shared `displayBrand()`, `turnOn()`, and `turnOff()` methods. It also declares public abstract void `operate()`; without providing one general implementation, because every appliance operates differently. WashingMachine and Refrigerator must therefore supply their own `operate()` methods.

In Main.java, both objects are held through Appliance references: `Appliance washingMachine = new WashingMachine("LG");` and a similar Refrigerator object. Calling `operate()` produces different behaviour, such as washing clothes or storing food and beverages, even though the calling pattern is the same. This combined abstraction with polymorphism and showed how a common interface can support different implementations.

This tutorial made OOP feel more useful in real systems. A future AirConditioner or Television class could extend Appliance and fit the existing Main program pattern without changing the basic power-control methods.

Weeks 8-9 - Collections, Loops, and File Persistence

The daily-task application connected several Java topics in one program. I used `ArrayList<String>` to store a flexible collection of tasks and `Scanner` to read three entries from the keyboard. A counting for loop controls the three input prompts, while another loop uses the list index to display numbered tasks. An enhanced for loop is then used when writing each task to the file. Comparing these loops helped me choose between working with an index and working directly with each value.

File I/O was the main new skill. `BufferedWriter` and `FileWriter` save the tasks to `task.txt`, and `BufferedReader` with `FileReader` loads each line and prints it. Both operations use try-with-resources, so the streams close automatically, and `IOException` is handled without causing an uncontrolled crash. The while condition (`line = reader.readLine()`) `!= null` taught me a common pattern for processing a text file until its end.

This activity showed the meaning of persistence: data can remain outside the program in a text file instead of existing only in memory. The present version saves new tasks and reads them back in the same run. A useful next improvement would be to load existing tasks when the application starts, before asking the user to add or edit items.

Week 10 - Event-Driven Programming with Swing

The final tutorial introduced a graphical quiz battle. `Questions.java` acts as the data model: it stores the question, two options, and correct answer as private fields, provides getter methods, and checks a selected answer through `isCorrect(String choice)`. `QuizBattleGUI` extends `JFrame` and implements `ActionListener`. It creates labels and buttons, sets their positions, adds them to the frame, and registers the same listener with both buttons.

The key change from console programs was that execution depends on an event. In `actionPerformed(ActionEvent e)`, the program identifies the clicked `JButton`, compares its text with the answer, and updates the result label. A correct answer increases score and disables both buttons, while a wrong answer allows another attempt. I learned to think about interface state and user actions rather than only a top-to-bottom sequence.

Separating `Questions` from the GUI also reinforced good design. The question logic is not mixed into every button definition, so it would be easier to add more questions or another interface later.

3. Challenges Encountered

Moving from sequential code to objects. The early programs were easy to follow because every statement was inside `main()`. Week 2 required me to decide what belonged in the `Student` class and what belonged in the main program. Constructors, fields, and instance methods initially felt like extra parts, but they were necessary to keep the object responsible for its own data and behaviour.

Understanding inheritance and constructor chaining. The use of `extends`, `super`, and overridden methods was more difficult than simple method calls. I had to distinguish what was inherited from the parent from what was added by the child. It was especially important to pass name and ID to the parent constructor instead of duplicating those fields in `Student`, `Lecturer`, or `Employee`-related classes.

Recognizing polymorphism at runtime. A statement such as `Person p1 = new Student(...)` contains two types, which was confusing at first. I needed to understand that the reference type controls what members are accessible at compile time, while the real object type determines which overridden `introduce()` method runs. The `Appliance` example later made the same idea clearer.

Handling files and checked exceptions. File operations added resources that might fail because a file can be missing, inaccessible, or affected by an I/O error. I also had to ensure that every task was written on its own line and that reading stopped correctly at the end of the file. The required `IOException` handling made the program longer but also safer.

Adapting to event-driven GUI logic. In the `Swing` activity, the main method only opens the window; the important logic runs later when a button is clicked. I had to connect each button to the listener, identify the source of the event, and change the label, score, and button state correctly. This required a different mental model from console input and output.

Maintaining a clear GitHub repository. The work was completed over several weeks, so consistent folders, filenames, commits, and documentation became important. A small naming or case difference can matter in Java and on Linux. I also learned that a repository is easier to assess when every tutorial is kept in one place and its purpose is explained clearly.

Why these challenges mattered Each difficulty appeared when the programs became more structured or more realistic. Solving them improved both my Java knowledge and my ability to organize development work.

4. How the Challenges Were Overcome

1. **Break each problem into responsibilities.** I separated the data class from the class containing `main()`, then checked which fields, constructors, and methods each class should own. This approach made the Student, Questions, and GUI code easier to reason about.
2. **Trace the execution using concrete objects.** For inheritance and polymorphism, I followed one object from construction to method call. Tracing a Student stored in a Person reference, and then comparing it with a Lecturer, made dynamic method selection clearer.
3. **Compare the program with the required output.** I used the expected results from each tutorial to check labels, values, loop numbering, and method behaviour. Compiler feedback, console output, and visible GUI state helped locate missing methods, incorrect names, and logic errors.
4. **Use safe language features and documentation.** Try-with-resources simplified file closing, catch blocks handled I/O problems, and comments explained the purpose of important code. The Week 5 documentation exercise also forced me to explain encapsulation in my own words instead of only copying a pattern.
5. **Save progress through GitHub.** Committing tutorial work in stages gave me a history of changes and reduced the risk of losing files. Reviewing the repository structure before submission helped me see the project as one complete portfolio rather than unrelated weekly exercises.

5. Improvements in Java Programming Skills

My syntax became more accurate and readable. I am now more comfortable with variable types, conditions, loops, method declarations, parameters, return values, constructors, and object creation. I also improved at choosing a loop for a purpose: a counting loop for a fixed number of inputs, an index loop for numbered display, an enhanced loop for each stored task, and a while loop for reading until the end of a file.

More importantly, my design skills improved. I can now separate a problem into related classes, protect fields, reuse parent behaviour, override methods, and define an abstract contract. The Questions and QuizBattleGUI classes showed me how separating data and interface logic can make a program easier to extend. I also gained introductory experience with collections, persistent files, exceptions, and event-driven programming.

The assignment improved non-coding skills as well. I practiced explaining technical decisions, organizing files, writing a README and supporting documentation, and maintaining work in GitHub. These habits make code easier to review and are relevant to team projects and future employment.

6. Understanding of Object-Oriented Programming Concepts

The tutorials showed that the main OOP concepts are connected rather than independent definitions. Encapsulation protects an object's state, inheritance reuses a common design, abstraction describes required behaviour, and polymorphism lets different objects respond through one common type. The repository contains a direct example of each concept.

Concept	Evidence in my code	My understanding
Class and object	student / s1; Questions / question	A class defines data and behaviour; an object is a working instance with its own state.
Encapsulation	Week 5 Student private fields and getters/setters	Access is controlled through methods, allowing protection and future validation.
Inheritance	Student and Lecturer extend Person; Lecturer extends Employee	A child reuses the parent's common state and behaviour, then adds specialization.
Polymorphism	Person references to Student/Lecturer; Appliance references	One parent type can represent several child objects and run the correct overridden method.
Abstraction	Abstract Appliance and operate()	A parent defines the common contract while each concrete appliance supplies its own operation.

7. Future Learning Plans

1. **Follow standard Java naming and package conventions.** I plan to use PascalCase consistently for class and file names, organize related classes into packages, and learn Maven or Gradle for repeatable builds.
2. **Add validation and stronger exception handling.** Future setters will reject invalid values such as an impossible CGPA, while applications will give users clearer recovery messages for invalid input and file problems.
3. **Learn automated testing with JUnit.** Unit tests for grade boundaries, Questions.isCorrect(), and student validation would make behaviour easier to verify after changes.
4. **Expand the task application.** I would load saved tasks at startup, allow add/edit/delete operations, avoid replacing existing data unintentionally, and later provide a simple GUI.
5. **Expand the quiz battle.** I plan to store multiple Questions in a collection, move to the next question automatically, show progress, calculate a final score, and add a restart option.
6. **Continue with advanced OOP and Git practice.** Interfaces, composition, generics, design patterns, branches, pull requests, and clearer commit messages are the next skills I want to develop through larger projects.

8. Conclusion

Completing these tutorials moved me from writing a few console statements to building programs made from cooperating classes. Week 1 established Java syntax and version-control basics. Week 2 introduced objects and constructors. Weeks 3-7 developed the core OOP principles through people, employees, students, and appliances. Weeks 8-9 added collections and persistent text files, and Week 10 introduced a graphical, event-driven application. Because each activity extended the previous knowledge, I could see my progress across the repository instead of treating each topic as an isolated definition.

The most valuable outcome is that I now understand why OOP structure matters. Encapsulation protects data, inheritance and abstraction reduce duplication, and polymorphism allows flexible behaviour. I also learned that readable code, documentation, folder organization, and Git history are part of a complete software project. I still need more practice with validation, automated testing, packages, build tools, and larger interfaces, but this assignment has given me a strong base for those next steps.

Final reflection My main improvement was not only learning more Java syntax; it was learning to design programs so that their data, responsibilities, and behaviour are organized clearly.

9. GitHub Repository URL

The complete tutorial source files and supporting materials reviewed for this reflection are available in the following public repository:

GITHUB REPOSITORY

github.com/mask2942002/

[SALEH_ANWAR_MOSLEH_ANWAR_202504010001_OOP_JAVA](https://github.com/mask2942002/SALEH_ANWAR_MOSLEH_ANWAR_202504010001_OOP_JAVA)

Source Note

Assignment brief: BIT1123 Object Oriented Programming - Assignment 1 Individual (20%), City University Malaysia, 2026.

Repository reviewed: [SALEH_ANWAR_MOSLEH_ANWAR_202504010001_OOP_JAVA](https://github.com/mask2942002/SALEH_ANWAR_MOSLEH_ANWAR_202504010001_OOP_JAVA)